WorkRadar — AI Workflow

How AI was used to design, build and document the MVP | October 2026

1. AI tools used

Tool	Used for
Claude (Opus 5.5, Claude app with code workspace)	Primary tool end to end: PRD, product decisions, mock data, the full React + TypeScript codebase, CSS design system, QA, design PDF and this document.
Claude API (Messages API, runtime, optional)	Inside the product: writes the AI Daily Brief and answers search questions from the already-ranked item context, when an API key is set in Settings.
Playwright + headless Chromium (driven by Claude)	Rendered every screen and state at desktop and mobile sizes for visual review and for the high-fidelity design PDF; scripted click-through tests of Assign, Done, Snooze, Undo and search.
esbuild, ReportLab	Bundling the app; generating the PRD, UX design and AI Workflow PDFs from Python scripts written by Claude.

2. Significant prompts

- **Product framing:** “Build WorkRadar, a web application that helps managers quickly understand top work for the day, what happened today, what requires immediate attention and who is blocked, without checking Slack, Jira, GitHub or Email”, followed by the PRD section list (max 2 pages).
- **Build:** the full brief: UX screens (Dashboard, Priority Feed, Item Details, Search, Empty/Error) and a working MVP with dashboard, AI summary, priority feed, search, detail page and Assign / Mark Done / Snooze, on mock data.
- **Runtime brief prompt** (system prompt shipped in *src/ai/claude.ts*, abridged):

```
You are WorkRadar, a chief-of-staff assistant for an engineering manager. You receive work items from Slack, Jira, GitHub and Email that have ALREADY been ranked by an explainable priority engine... Merge items that describe the same issue across tools into one bullet. Never invent facts, people, numbers or items. Every bullet must cite the ids of the items it is based on in "itemIds". Sections: attention (max 3), blocked (max 4), today (max 3), happened (max 2). Respond with JSON only.
```

- **Runtime search prompt:** “Answer the engineering manager’s question in 1–3 sentences using ONLY the provided work items. Cite refs like PAY-482. If the items do not answer it, say so.”

3. What AI generated

Deliverable	Generated by AI
PRD	Problem framing, three personas, before/after user journey, measurable success metrics, scope, assumptions and the must-have vs nice-to-have split.
Mock data	A realistic 8-person payments squad and 30 cross-tool items (an incident with its Jira bug, hotfix PR and customer escalation; blockers; review bottlenecks; shipped work), with timestamps relative to load time so the demo always looks like “this morning”.
AI engine	Explainable priority scoring (each point has a reason), blocker detection (status, language cues, review age, silence), cross-tool de-duplication, local brief generator, next-step and assignee suggestions, natural-language search parser, Claude client with validation and fallback.
UI	All pages and components, icon set, design tokens (light + dark), responsive layout, toasts with Undo, loading / empty / partial-error / outage states and the demo-state switcher.
Design & docs	Screenshot pipeline, annotated 12-page UX design PDF, README, this document.

4. What was reviewed and modified

Every screen was rendered and inspected after each build; the following changes came out of that review rather than the first draft:

- **Blocker accuracy:** the same blocker reported in Slack and Jira (and a Slack complaint about a stale PR) appeared twice; grouping was changed to merge by “waiting on” person and prefer the Jira/GitHub record. Blocked durations had been taken from the oldest unrelated item and now use the primary blocker.
- **Brief wording:** the headline (“3 critical items, 4 high-priority items and…”) was rewritten to “7 items need your attention (3 critical) and 3 teammates are blocked. Start with…”; duplicated “Linked: …” text was removed because source chips already show it.
- **Search:** “github prs waiting for review” returned nothing because “waiting” was treated as a required keyword; review words now map to the In-review status and filler words are ignored.
- **Assign suggestion:** the first version suggested the QA engineer for a backend story (high overlap, low load); QA is now only suggested for QA-labelled work.
- **Layout:** the cramped “Linked across tools” rows were simplified, redundant “4.1d · 4.1d” durations removed, and the sidebar background fixed on long pages.
- **Tooling:** the npm registry was unreachable in the build environment, so the planned Vite setup was replaced with a dependency-light esbuild build (React only) that ships a zero-install standalone HTML file.

5. Key trade-offs

Decision	Why	Cost
Deterministic ranking + LLM narrative	Rankings are explainable, testable and stable; the LLM only summarises and must cite item ids.	Weights are hand-tuned; a learned model could adapt better to each team.
Local AI engine as default and fallback	Works offline, with no key or cost, and the dashboard never breaks if the LLM fails.	Template summaries are less fluent than LLM prose.
Mock data in a unified schema	Fast to build and demo; adapters can be swapped for real APIs without UI changes.	No proof of real-world volume, noise or permission edge cases.
Browser-direct Claude call	Lets reviewers try the LLM mode with zero backend.	Exposes the key client-side; production must proxy through a server.
localStorage for actions	Assign / Done / Snooze persist with no backend; Undo is trivial.	Single-user, single-device; not written back to Jira or Slack.
No individual performance metrics	Keeps the product about unblocking teams, not surveillance; builds trust.	Directors may still ask for per-person analytics later.
No UI or icon libraries	Small bundle, full design control, nothing to install offline.	More hand-written CSS and components to maintain.